

Arquitetura Técnica para Sistemas de Agentes Autônomos Hierárquicos em Ambiente Windows Subsystem for Linux

A evolução da inteligência artificial agentiva exige uma transição de simples chamadas de modelos de linguagem para ecossistemas complexos de agentes que colaboram de forma autônoma. No contexto de um ambiente Windows Subsystem for Linux (WSL), a arquitetura deve ser desenhada para maximizar a performance nativa do Linux enquanto mantém a interoperabilidade com o sistema operativo anfitrião. Este blueprint técnico detalha uma estrutura de sete agentes, orquestrados de forma hierárquica, focando-se em robustez, memória de longo prazo e segurança operacional.

Análise de Frameworks de Orquestração de Agentes

A escolha do framework de orquestração define a topologia de comunicação e a gestão de estado do sistema. Três paradigmas principais dominam o cenário atual: LangGraph, AutoGen e CrewAI. Cada um oferece abordagens distintas para a coordenação de tarefas e a manutenção do contexto.¹

LangGraph destaca-se em fluxos de trabalho cíclicos e complexos, tratando as interações entre agentes como nós num gráfico direcionado.¹ Esta abordagem é ideal para processos que exigem lógica condicional persistente e a capacidade de retroceder ou repetir etapas com base em novos dados, uma característica essencial para o desenvolvimento de software e automação de sistemas onde erros são frequentes.⁴ A sua integração com o ecossistema LangChain e ferramentas como o LangSmith permite uma observabilidade profunda, facilitando a depuração de comportamentos emergentes em sistemas de múltiplos agentes.²

CrewAI foca-se numa colaboração baseada em papéis, onde cada agente é tratado como um membro de uma equipa com responsabilidades e objetivos específicos.³ Embora seja intuitivo para processos sequenciais ou lineares que mimetizam estruturas organizacionais humanas, pode tornar-se rígido em cenários que exigem mudanças dinâmicas de estado ou ciclos de retroalimentação complexos.¹ Por outro lado, AutoGen facilita conversas dinâmicas entre agentes, permitindo que o fluxo de trabalho emergja do diálogo.¹ No entanto, a gestão de contexto em AutoGen pode sofrer de "bloat" informacional, uma vez que a memória é muitas vezes baseada no histórico de transcrição da conversa, o que exige técnicas agressivas de poda à medida que a janela de contexto se aproxima dos limites.²

Para um sistema hierárquico de sete agentes, a aplicação do "Supervisor Pattern" via LangGraph revela-se a mais viável. Neste modelo, um Agente Mestre (Supervisor) atua como o nó central de decisão, recebendo entradas, avaliando o progresso e delegando subtarefas

aos agentes subordinados através de arestas condicionais.⁵ Esta estrutura garante que o Agente Mestre mantenha uma visão global do sistema, enquanto os agentes especialistas operam em domínios isolados, reduzindo a carga cognitiva individual e prevenindo a diluição do foco.⁵

Comparação de Capacidades de Orquestração

Característica	LangGraph	AutoGen	CrewAI
Paradigma de Controlo	Grafo de Estados (Cíclico)	Conversacional (Dinâmico)	Baseado em Papéis (Estruturado)
Gestão de Estado	Persistente com Checkpoints	Histórico de Chat	Baseado em Saídas de Tarefa
Complexidade de Implementação	Alta (Curva de aprendizagem acentuada)	Média	Baixa (YAML/Python intuitivo)
Adequação para Ciclos	Excelente	Moderada	Limitada
Escalabilidade	Alta (Sistemas distribuídos)	Moderada	Moderada

A capacidade de LangGraph para gerir estados persistentes através de "checkpointers" (como SQLite ou PostgreSQL) permite que o sistema de agentes retome operações após interrupções, uma funcionalidade crítica para fluxos de trabalho que podem durar semanas ou meses.³ No ambiente WSL, a utilização de bases de dados locais Linux para esta persistência garante baixa latência e alta integridade de dados.³

Estratégia de Memória Hierárquica de Longo Prazo

A gestão de memória é o maior desafio técnico para agentes que operam de forma contínua. Os sistemas tradicionais de Retrieval-Augmented Generation (RAG) dependem de bases de dados vetoriais simples, mas sofrem de "distração de contexto" quando a escala da informação aumenta.⁸ Para evitar o estouro da janela de contexto e manter a relevância, propõe-se uma arquitetura de Memória Hierárquica (H-MEM).

Estrutura de Camadas H-MEM

A H-MEM organiza a informação em quatro níveis de abstração semântica, permitindo uma recuperação direcionada em vez de exaustiva.⁹ Esta organização assemelha-se à estrutura de um sistema de arquivos ou de um documento complexo, facilitando a navegação do agente pela sua própria história.⁸

1. **Camada de Domínio:** O nível mais alto, identificando áreas amplas de conhecimento (ex: Engenharia, Segurança, Administração de Sistemas).⁹
2. **Camada de Categoria:** Subdomínios específicos (ex: Python para WSL, Configuração de Rede Ubuntu).⁹
3. **Camada de Traço de Memória:** Resumos de threads de conversação ou cadeias de raciocínio passadas.⁸
4. **Camada de Episódio:** O conteúdo bruto das interações, incluindo comandos executados, logs de erro e preferências do utilizador.⁸

A recuperação de informação segue um processo de "routing" baseado em índices. Em vez de comparar uma consulta contra milhões de vetores, o sistema utiliza codificação de índice de auto-posicionamento para navegar pelas camadas.⁸ A consulta é primeiro comparada com a Camada de Domínio; uma vez identificado o domínio, os ponteiros de índice guiam a busca apenas para as categorias e episódios subordinados.⁹ Este método reduz a complexidade computacional de $O(N)$ para aproximadamente $O(\log N)$, resultando em economias de tokens superiores a 90% e reduções significativas na latência de resposta.⁸

Implementação de Bases de Dados Vetoriais: Milvus vs. Pinecone

A escolha entre Milvus e Pinecone para o ambiente WSL depende da necessidade de soberania de dados e latência. Milvus é uma base de dados vetorial de código aberto, otimizada para infraestrutura local, que suporta esquemas de metadados complexos necessários para a filtragem escalar exigida pela H-MEM.¹⁰ Pinecone, sendo um serviço gerido na nuvem, oferece escalabilidade simplificada mas introduz latência de rede e dependência de serviços externos, o que pode comprometer a automação de sistemas sensíveis ao tempo.¹²

Característica	Milvus (Local/WSL)	Pinecone (Cloud)
Localização de Dados	Interna ao Ubuntu/WSL	Servidores Externos
Latência de Recuperação	Baixa (Sub-milissegundo local)	Variável (Dependente de rede)
Escalabilidade	Vertical/Horizontal	Serverless (Managed)

	(Self-managed)	
Esquema de Metadados	Altamente Flexível	Suporta Filtros Básicos

No contexto de sete agentes, Milvus permite a criação de coleções isoladas para diferentes agentes ou domínios, facilitando o isolamento de dados e a manutenção da integridade da memória.¹⁰ A integração de técnicas de indexação como HNSW (Hierarchical Navigable Small World) garante que a busca por similaridade permaneça eficiente mesmo com milhões de pontos de dados.⁹

Camada de Destilação de Contexto

A "Context Distillation" atua como um mecanismo de consolidação, transformando experiências brutas em memórias estruturadas.¹⁴ Periodicamente, um Agente de Custódia de Memória analisa os episódios recentes e extrai factos, decisões de design e lições aprendidas.⁹ Este processo utiliza um modelo de linguagem para comprimir longas sequências de logs em "Traços de Memória" semânticos, que são então re-indexados nas camadas superiores da H-MEM.⁹ Esta compressão impede a degradação da performance do modelo provocada pela presença de ruído informativo na janela de contexto.⁸

Integração Dinâmica de APIs e Otimização de Custos

Um sistema de agentes eficiente deve ser capaz de selecionar o modelo de inteligência artificial mais adequado para cada tarefa específica. Esta estratégia, gerida através de uma gateway unificada como o LiteLLM, permite alternar entre modelos proprietários de alto desempenho e modelos locais de custo zero.¹⁶

Hierarquia de Modelos e Especialização

O Agente Mestre utiliza uma lógica de roteamento baseada na complexidade da tarefa e nos requisitos de precisão. Tarefas que exigem raciocínio abstrato profundo ou planeamento de longo prazo são enviadas para o GPT-4o, enquanto tarefas de programação intensiva e revisão de código beneficiam da precisão técnica do Claude 3.5 Sonnet.¹⁸

Para tarefas triviais, como a formatação de logs, triagem inicial de erros ou tradução de comandos simples, o sistema utiliza o Ollama para executar modelos locais como Llama 3.1 ou Mistral dentro do WSL.¹⁶ Esta abordagem não só reduz os custos operacionais, mas também garante a continuidade do sistema em caso de falhas na rede externa ou limitações de taxa (rate limits) nas APIs proprietárias.¹⁶

Modelo	Especialidade	Custo	Localização
--------	---------------	-------	-------------

Claude 3.5 Sonnet	Codificação, Raciocínio Técnico	Médio/Alto	API Externa
GPT-4o	Raciocínio Geral, Visão, Multimodal	Médio/Alto	API Externa
Llama 3.1 (via Ollama)	Tarefas Triviais, Triagem, Resumo	Zero (Local)	WSL (Local)
Claude 3.5 Haiku	Respostas Rápidas, Baixa Latência	Baixo	API Externa

Implementação com LiteLLM e Fallbacks

O LiteLLM atua como um "proxy" universal que traduz as solicitações dos agentes para o formato específico de cada fornecedor.¹⁶ A configuração permite a definição de cadeias de fallback: se o Claude 3.5 Sonnet falhar ou atingir o limite de quota, o sistema pode automaticamente reencaminhar a tarefa para o GPT-4o ou, em última instância, para um modelo local potente.¹⁶

A monitorização do uso de tokens e dos tempos de resposta em tempo real, facilitada pelo LiteLLM e ferramentas de observabilidade associadas, permite ao Agente Mestre ajustar a estratégia de roteamento dinamicamente para manter o equilíbrio entre performance e orçamento.¹⁶ A utilização de cache em Redis para respostas frequentes e a implementação de chamadas em lote (batching) via JSON-RPC contribuem ainda mais para a redução da latência global do sistema.¹⁶

Automação Segura do Sistema Operativo no WSL

Conceder a agentes autónomos acesso ao terminal e ao sistema de arquivos de um ambiente WSL2 requer camadas rigorosas de segurança. A arquitetura proposta implementa o padrão "The Auditor", um agente de verificação independente que atua como um guardião entre os agentes de execução e o sistema operativo.²¹

O Padrão "The Auditor" e Sandboxing

O Agente Auditor intercepta todos os comandos bash gerados pelo Agente de Desenvolvimento Full-Stack antes de serem submetidos ao terminal.²¹ Este agente opera sob um conjunto de regras de permissão estritas, definidas em ficheiros de configuração geridos pelo administrador do sistema.²²

O sistema de permissões utiliza uma sintaxe hierárquica para permitir, negar ou solicitar confirmação humana para comandos específicos ²²:

- **Permitir (Allow):** Comandos seguros e frequentes, como `ls`, `git status` ou `npm test`.²²
- **Negar (Deny):** Comandos destrutivos ou de exfiltração de dados sensíveis, como `rm -rf /`, `curl` para domínios desconhecidos ou leitura de ficheiros `.env`.²²
- **Perguntar (Ask):** Comandos que alteram permanentemente o estado do sistema ou do repositório, como `git push`, instalações de pacotes globais ou modificações em configurações de rede.²²

Para reforçar a segurança, a execução de comandos ocorre dentro de uma "sandbox" de bash no WSL2.²² Esta sandbox isola o ambiente de execução, limitando o acesso a diretórios específicos e restringindo a conectividade de rede apenas a domínios pré-aprovados.²² A utilização do plugin do Microsoft Defender para WSL garante que as atividades do subsistema Linux sejam monitorizadas pelo motor de segurança do Windows host, proporcionando uma camada adicional de deteção de ameaças.²³

Protocolo de Comunicação e Validação de Intenção

A comunicação entre o Agente de Desenvolvimento e o Auditor baseia-se no Model Context Protocol (MCP), utilizando esquemas JSON-RPC para garantir que as intenções sejam transmitidas de forma estruturada.²¹ Isto elimina a ambiguidade da linguagem natural, forçando o agente a especificar claramente o comando, os argumentos e a justificação para a ação.²¹

A verificação independente realizada pelo Auditor envolve:

1. **Validação de Esquema:** Garantir que o pedido segue a estrutura JSON-RPC correta.²¹
2. **Análise de Risco:** Avaliar o potencial impacto do comando no sistema de arquivos e na integridade do SO.²²
3. **Controlo de Propriedade de Recursos:** Assegurar que apenas um agente tem permissão para modificar um recurso específico num determinado momento, evitando conflitos de escrita.²¹

Ciclo de Self-Healing e Protocolo de Execução

A robustez de um agente autónomo reside na sua capacidade de recuperar de erros de execução sem supervisão constante. O protocolo "Try-Heal-Retry" define a interação entre o Agente de Desenvolvimento Full-Stack e o terminal para garantir a resolução autónoma de problemas.²⁶

Arquitetura Try-Heal-Retry

Quando um comando executado no terminal WSL resulta num erro (código de saída diferente

de zero), o sistema inicia automaticamente o ciclo de self-healing ²⁶:

1. **Fase de Tentativa (Try):** O agente executa a tarefa inicial. Se for bem-sucedida, o fluxo prossegue normalmente.
2. **Fase de Cura (Heal):** Em caso de falha, o sistema captura o "crime scene" — o rastreo de erro (traceback), as últimas linhas do log de execução e uma amostra dos ficheiros relevantes.²⁶ Esta informação é enviada para um Agente de Cura (muitas vezes o próprio Agente de Desenvolvimento com um prompt de diagnóstico especializado) que analisa as causas raízes.²⁶
3. **Fase de Re-execução (Retry):** O agente propõe uma correção (ex: ajustar um delimitador num script, corrigir uma dependência em falta ou alterar permissões de um ficheiro) e tenta novamente a execução.²⁶

O protocolo estipula um limite de três tentativas autónomas.²⁶ Se, após a terceira tentativa, o erro persistir, o Agente de Desenvolvimento deve interromper o ciclo e escalar o problema para o Agente Mestre, fornecendo um resumo detalhado das tentativas falhadas e das possíveis razões para a insolubilidade do problema.²⁶

Implementação Técnica de Retentativas

A lógica de retentativa é implementada utilizando bibliotecas como "Tenacity" em Python, que permitem gerir de forma limpa os intervalos entre tentativas e as condições de paragem.²⁶ A utilização de Pydantic garante que as correções sugeridas pelo LLM sejam convertidas em objetos de dados estruturados, prontos para serem integrados na próxima chamada de terminal.²⁶

Componente de Retry	Função	Exemplo de Implementação
Estratégia de Backoff	Aumentar o tempo entre tentativas	Exponencial com Jitter ²⁷
Classificação de Erros	Distinguir erros temporais de fatais	Retentar HTTP 5xx, não retentar 4xx ²⁸
Persistência de Estado	Guardar progresso antes do retry	Checkpoints no LangGraph ⁵
Idempotência	Garantir resultados consistentes	Verificação de estado antes de re-executar ²⁷

Para manter a integridade do sistema, o Agente Mestre pode optar por reverter o estado do ambiente (time-travel) para um checkpoint anterior estável antes de tentar uma abordagem completamente diferente, utilizando as capacidades nativas de rollback do LangGraph.⁵

Blueprint Técnico da Arquitetura

O sucesso da implementação deste sistema depende da integração harmoniosa de uma stack tecnológica robusta e de protocolos de comunicação eficientes.

Recomendação da Stack Tecnológica

A stack recomendada para um ambiente WSL (Ubuntu) otimiza a performance local enquanto aproveita os melhores modelos de IA disponíveis através de APIs.

Camada	Tecnologia	Justificação
Orchestrator	LangGraph	Suporte nativo para grafos cíclicos, persistência de estado e hierarquia. ¹
Vector DB	Milvus	Alta performance local, suporte a esquemas complexos para H-MEM. ¹⁰
API Gateway	LiteLLM	Gestão centralizada de modelos, fallbacks e otimização de custos. ¹⁶
Local Inference	Ollama	Execução eficiente de modelos de código aberto no WSL. ¹⁶
Communication	MCP (JSON-RPC)	Protocolo estruturado para colaboração inter-agente segura. ²¹
Security	Claude Code Sandbox	Ambiente isolado e configurável para automação de terminal. ²²
Self-Healing	Tenacity + Pydantic	Lógica de retentativa robusta com validação de

		tipos. ²⁶
--	--	----------------------

Esquema de Base de Dados para Memória Hierárquica

O esquema no Milvus deve ser desenhado para suportar a navegação em camadas da H-MEM.

1. **Coleção de Domínios:** Vetores de alta dimensão representando áreas temáticas globais. Metadados: domain_id, domain_name, index_pointers.
2. **Coleção de Categorias:** Filtros escalares associados a cada domínio. Metadados: category_id, parent_domain_id, category_tags.
3. **Coleção de Episódios:** Armazenamento denso de vetores e texto bruto. Metadados: episode_id, timestamp, agent_role, session_id, success_metric.

Protocolo de Comunicação Inter-Agentes

A coordenação entre os sete agentes segue o modelo JSON-RPC 2.0 sobre o Model Context Protocol. Cada agente possui um "Manifesto de Capacidades" que informa o Agente Mestre sobre quais os métodos e ferramentas que pode executar.²¹ As mensagens são tipificadas como request, inform, commit ou reject, permitindo um fluxo de trabalho claro e auditável.²¹

Este blueprint fornece a base para um sistema de agentes resiliente, capaz de operar em ciclos de desenvolvimento complexos no WSL, mantendo a segurança operacional e a relevância contextual ao longo de extensos períodos de atividade autónoma. A integração de memória hierárquica e self-healing transforma o sistema de uma simples ferramenta de automação num ecossistema inteligente de resolução de problemas.

Referências citadas

1. LangGraph vs AutoGen vs CrewAI: Complete AI Agent Framework ..., acessado em fevereiro 7, 2026, <https://latenode.com/blog/platform-comparisons-alternatives/automation-platform-comparisons/langgraph-vs-autogen-vs-crewai-complete-ai-agent-framework-k-comparison-architecture-analysis-2025>
2. CrewAI vs AutoGen vs LangGraph: Top Multi-Agent Frameworks for 2026 - DataMites, acessado em fevereiro 7, 2026, <https://datamites.com/blog/crewai-vs-autogen-vs-langgraph-top-multi-agent-frameworks/>
3. CrewAI vs LangGraph vs AutoGen: Choosing the Right Multi-Agent ..., acessado em fevereiro 7, 2026, <https://www.datacamp.com/tutorial/crewai-vs-langgraph-vs-autogen>
4. OpenAI Agents SDK vs LangGraph vs Autogen vs CrewAI - Composio, acessado em fevereiro 7, 2026, <https://composio.dev/blog/openai-agents-sdk-vs-langgraph-vs-autogen-vs-crewai>

5. LangGraph: Agent Orchestration Framework for Reliable AI Agents, acessado em fevereiro 7, 2026, <https://www.langchain.com/langgraph>
6. AutoGen vs. CrewAI vs. LangGraph vs. OpenAI Multi-Agents Framework - Galileo AI, acessado em fevereiro 7, 2026, <https://galileo.ai/blog/autogen-vs-crewai-vs-langgraph-vs-openai-agents-framework>
7. LangGraph Tutorial: What Is LangGraph and How to Use It ..., acessado em fevereiro 7, 2026, <https://www.datacamp.com/tutorial/langgraph-tutorial>
8. How to Design Efficient Memory Architectures for Agentic AI Systems ..., acessado em fevereiro 7, 2026, <https://pub.towardsai.net/how-to-design-efficient-memory-architectures-for-agentic-ai-systems-81ed456bb74f>
9. Hierarchical Memory for High-Efficiency Long-Term ... - arXiv, acessado em fevereiro 7, 2026, <https://arxiv.org/abs/2507.22925>
10. Hierarchical Document Search System with Milvus Vector Database - Alok Sahu - Medium, acessado em fevereiro 7, 2026, <https://alokkrsahu.medium.com/hierarchical-document-search-system-with-milvus-vector-database-5e1df7d718d0>
11. Ultimate Guide to Vector Databases - Dataaspirant, acessado em fevereiro 7, 2026, <https://dataaspirant.com/vector-database/>
12. Pinecone: Vector Database for Scalable AI Search, acessado em fevereiro 7, 2026, <https://thirdeyedata.ai/pinecone-vector-search/>
13. How to Implement Vector Databases - OneUptime, acessado em fevereiro 7, 2026, <https://oneuptime.com/blog/post/2026-01-25-vector-databases/view>
14. MemVerse: Multimodal Memory for Lifelong Learning Agents - arXiv, acessado em fevereiro 7, 2026, <https://arxiv.org/html/2512.03627v1>
15. Comparing Memory Systems for LLM Agents: Vector, Graph, and Event Logs, acessado em fevereiro 7, 2026, <https://www.marktechpost.com/2025/11/10/comparing-memory-systems-for-llm-agents-vector-graph-and-event-logs/>
16. litellm integration · ruvnet/claude-flow Wiki · GitHub, acessado em fevereiro 7, 2026, <https://github.com/ruvnet/claude-flow/wiki/litellm-integration>
17. LiteLLM Integration - Bytebot - Self-Hosted AI Desktop Agent, acessado em fevereiro 7, 2026, <https://docs.bytebot.ai/deployment/litellm>
18. Comparing GPT-4o, LLaMA 3.1, and Claude 3.5 Sonnet - Walturn, acessado em fevereiro 7, 2026, <https://www.walturn.com/insights/comparing-gpt-4o-llama-3-1-and-claude-3-5-sonnet>
19. Comparison Analysis: Claude 3.5 Sonnet vs GPT-4o - Vellum AI, acessado em fevereiro 7, 2026, <https://www.vellum.ai/blog/claude-3-5-sonnet-vs-gpt4o>
20. Pros and Cons of JSON-RPC and REST APIs Protocols - CoinAPI.io, acessado em fevereiro 7, 2026, <https://www.coinapi.io/blog/pros-and-cons-of-json-rpc-and-rest-apis-protocols>
21. Why Multi-Agent LLM Systems Fail (and How to Fix Them ..., acessado em fevereiro 7, 2026,

<https://www.augmentcode.com/guides/why-multi-agent-llm-systems-fail-and-how-to-fix-them>

22. Claude Code settings - Claude Code Docs, acessado em fevereiro 7, 2026,
<https://code.claude.com/docs/en/settings>
23. Microsoft Defender for Endpoint plug-in for Windows Subsystem for Linux (WSL),
acessado em fevereiro 7, 2026,
<https://learn.microsoft.com/en-us/defender-endpoint/mde-plugin-wsl>
24. MCP API Gateway Explained: Protocols, Caching, and Remote Server Integration -
Gravitee, acessado em fevereiro 7, 2026,
<https://www.gravitee.io/blog/mcp-api-gateway-explained-protocols-caching-and-remote-server-integration>
25. AI4CyberMLHat_2022/technique_short_descriptions.json at main · ALFA-group ...
- GitHub, acessado em fevereiro 7, 2026,
https://github.com/ALFA-group/AI4CyberMLHat_2022/blob/main/technique_short_descriptions.json
26. Building a Self-Healing Data Pipeline That Fixes Its Own Python Errors, acessado
em fevereiro 7, 2026,
<https://towardsdatascience.com/building-a-self-healing-data-pipeline-that-fixes-its-own-python-errors/>
27. 5 Recovery Strategies for Multi-Agent LLM Failures | newline - Fullstack.io,
acessado em fevereiro 7, 2026,
<https://www.newline.co/@zaoyang/5-recovery-strategies-for-multi-agent-llm-failures--673fe4c4>
28. Mastering Retry Logic Agents: A Deep Dive into 2025 Best Practices - Sparkco,
acessado em fevereiro 7, 2026,
<https://sparkco.ai/blog/mastering-retry-logic-agents-a-deep-dive-into-2025-best-practices>
29. Getting Started - LiteLLM Docs, acessado em fevereiro 7, 2026,
<https://docs.litellm.ai/docs/>